C++ RAII

and Resource Management

Interview Questions & Answers with Code Examples

RAII Fundamentals & Exception Safety | Rule of Three / Five / Zero
Smart Pointers: unique_ptr, shared_ptr, weak_ptr
Scope Guards, Lock Management & Mutex Types
Move Semantics, Perfect Forwarding & noexcept
Memory Ownership Patterns & pimpl Idiom
Advanced RAII: Custom Deleters, Transactions, Threads
Pitfalls: Cycles, Zombies, Double-free, Anti-Patterns
Modern C++17/20: optional, coroutines, jthread, pmr

50 Expert Questions | C++11 through C++23

SECTION 1 — RAII Fundamentals

Q1. What is RAII and what problem does it solve? [Core Concept]

Answer

RAII (Resource Acquisition Is Initialization) is a C++ programming idiom where resource ownership is bound to object lifetime. Resources are acquired in the constructor and released unconditionally in the destructor. This solves the problem of resource leaks caused by early returns, exceptions, or programmer error — the destructor ALWAYS runs when an object goes out of scope, even if an exception is thrown.

Example

```
// WITHOUT RAII — leak-prone:
void withoutRAII() {
    FILE* f = fopen("data.txt", "r");
    if (!f) return;
    // ... process ...
    if (someError()) return; // LEAK: fclose never called!
    fclose(f);
}

// WITH RAII — leak-proof:
class FileGuard {
    FILE* f_;
public:
    explicit FileGuard(const char* name)
        : f_(fopen(name, "r")) {
        if (!f_) throw std::runtime_error("cannot open");
    }
    ~FileGuard() { if (f_) fclose(f_); } // Always runs!
    FILE* get() const { return f_; }
};

void withRAII() {
    FileGuard fg("data.txt"); // acquire
    if (someError()) return;  // destructor runs: file closed
    // destructor runs at end of scope too
}
```

Q2. Why is RAII exception-safe by definition? [Exception Safety]

Answer

C++ guarantees that destructors of all fully-constructed objects in scope are called during stack unwinding when an exception propagates. Since RAII wraps the resource in an object, the destructor fires regardless of whether the scope exits normally or via exception. This is the only reliable way to achieve exception-safe resource management in C++ without try/finally (which C++ intentionally omits).

Example

```
class MutexLock {
    std::mutex& m_;
public:
    explicit MutexLock(std::mutex& m) : m_(m) { m_.lock(); }
    ~MutexLock() { m_.unlock(); } // runs on ANY exit
};

std::mutex g_mutex;

void exceptionSafeWork() {
    MutexLock lock(g_mutex); // mutex locked
    doWork();                // might throw
```

```
    processMore();          // might throw
    // mutex ALWAYS unlocked here — normal return OR exception
}
// Without RAII: exception -> mutex left locked -> deadlock!
```

Q3. What are the four levels of exception safety and how does RAII help achieve them? [Exception Safety]

Answer

No-throw guarantee: the operation never throws. Strong guarantee: if an exception is thrown the program state is unchanged (commit-or-rollback). Basic guarantee: no resources are leaked and invariants are preserved, but state may change. No guarantee: anything may happen. RAII directly provides the basic guarantee (no leaks) and, combined with copy-and-swap, enables the strong guarantee.

Example

```
// Strong guarantee via copy-and-swap idiom
class Buffer {
    std::unique_ptr<char[]> data_;
    size_t size_;
public:
    Buffer(size_t n) : data_(std::make_unique<char[]>(n)), size_(n) {}

    // Strong guarantee: swap only if assignment succeeds
    Buffer& operator=(Buffer other) { // copy made in param
        std::swap(data_, other.data_); // no-throw swap
        std::swap(size_, other.size_);
        return *this;
    } // old data destroyed in other's dtor
};
```

Q4. What resources can RAII manage beyond memory? [Core Concept]

Answer

RAII is not limited to memory. It can manage any resource that needs deterministic cleanup: file handles, network sockets, database connections, mutexes and locks, GPU resources (textures, command buffers), Windows HANDLE objects, COM references, transaction scopes, temporary files, SSL contexts, thread handles, and custom cleanup actions (scope guards).

Example

```
// Managing a socket
class Socket {
    int fd_;
public:
    explicit Socket(const char* host, int port)
        : fd_(::socket(AF_INET, SOCK_STREAM, 0)) {
        if (fd_ < 0) throw std::runtime_error("socket failed");
        // connect...
    }
    ~Socket() { if (fd_ >= 0) ::close(fd_); }
    Socket(const Socket&) = delete;
    Socket& operator=(const Socket&) = delete;
};

// Managing a database transaction
class Transaction {
    DB& db_;
    bool committed_ = false;
public:
    explicit Transaction(DB& db) : db_(db) { db_.begin(); }
```

```
    void commit() { db_.commit(); committed_ = true; }
    ~Transaction() { if (!committed_) db_.rollback(); }
};
```

Q5. What is the Rule of Three in the context of RAII? [Rule of Three/Five]

Answer

If a class manages a resource (and therefore defines a destructor), it almost certainly needs to define copy constructor and copy assignment operator too. Without them, the compiler generates shallow copies — two objects point to the same resource, and when the first destructor runs it frees the resource, leaving the second with a dangling pointer. The Rule of Three says: define all three or none.

Example

```
class Buffer {
    char* data_;
    size_t size_;
public:
    // Constructor: acquire
    explicit Buffer(size_t n)
        : data_(new char[n]), size_(n) {}

    // Destructor: release
    ~Buffer() { delete[] data_; }

    // Copy constructor: deep copy (Rule of Three)
    Buffer(const Buffer& o)
        : data_(new char[o.size_]), size_(o.size_) {
        std::copy(o.data_, o.data_ + o.size_, data_);
    }

    // Copy assignment: deep copy (Rule of Three)
    Buffer& operator=(Buffer o) { // copy-and-swap
        std::swap(data_, o.data_);
        std::swap(size_, o.size_);
        return *this;
    }
};
```

Q6. What is the Rule of Five and how does it extend the Rule of Three? [Rule of Three/Five]

Answer

C++11 added move semantics. If a class defines any of the five special members (destructor, copy constructor, copy assignment, move constructor, move assignment), it should define all five. The move constructor and move assignment operator enable transferring resource ownership without copying — critical for RAII classes managing expensive resources like dynamic memory or file handles.

Example

```
class Buffer {
    char* data_; size_t size_;
public:
    explicit Buffer(size_t n)
        : data_(new char[n]), size_(n) {}
    ~Buffer() { delete[] data_; }

    // Copy (deep)
    Buffer(const Buffer& o)
        : data_(new char[o.size_]), size_(o.size_)
    { std::copy(o.data_, o.data_+o.size_, data_); }
```

```
    Buffer& operator=(Buffer o)
    { std::swap(data_,o.data_); std::swap(size_,o.size_);
      return *this; }

    // Move (transfer ownership — O(1), no allocation)
    Buffer(Buffer&& o) noexcept
        : data_(o.data_), size_(o.size_)
    { o.data_ = nullptr; o.size_ = 0; }

    Buffer& operator=(Buffer&& o) noexcept {
        if (this != &o) {
            delete[] data_;
            data_ = o.data_; size_ = o.size_;
            o.data_ = nullptr; o.size_ = 0;
        }
        return *this;
    }
};
```

Q7. What is the Rule of Zero and when should you use it? [Rule of Three/Five]

Answer

The Rule of Zero says: if you can design your class so that it does not manage any resources directly (by using RAII members like unique_ptr, shared_ptr, string, or vector), you should define NONE of the five special members. The compiler-generated versions will correctly deep-copy, move, and destroy all members. This is the preferred modern C++ approach.

Example

```
// Rule of Zero: let members manage themselves
class Person {
    std::string name_;           // RAII
    std::vector<int> scores_;    // RAII
    std::unique_ptr<Address> addr_; // RAII
public:
    Person(std::string n, std::unique_ptr<Address> a)
        : name_(std::move(n)), addr_(std::move(a)) {}

    // No destructor, no copy, no move defined!
    // Compiler generates all correctly:
    //   dtor:   calls string/vector/unique_ptr dtors
    //   copy:   deep-copies string+vector, unique_ptr deleted
    //   move:   moves all members
};

// Explicitly document the intention:
Person p1("Alice", std::make_unique<Address>("NY"));
Person p2 = std::move(p1); // moves correctly
```

SECTION 2 — Smart Pointers

Q8. What is std::unique_ptr and how does it implement RAII? [unique_ptr]

Answer

std::unique_ptr is a move-only RAII wrapper for heap-allocated objects or arrays. It owns the resource exclusively — there is exactly one owner at all times. The destructor calls delete (or delete[] for arrays) or a custom deleter. It has zero overhead over a raw pointer because it is non-copyable and non-reference-counted. Always use make_unique instead of new to avoid exception-unsafe construction.

Example

```
#include <memory>

// Basic usage
auto p = std::make_unique<int>(42);
std::cout << *p;   // 42
// p goes out of scope -> delete called automatically

// Array ownership
auto arr = std::make_unique<int[]>(10);
arr[3] = 99;

// Transfer ownership (move, not copy)
auto p2 = std::move(p); // p is now null
std::cout << (p == nullptr); // 1

// Custom deleter (for C APIs)
auto file = std::unique_ptr<FILE, decltype(&fclose)>(
    fopen("data.txt","r"), fclose);

// Return from factory — no overhead
std::unique_ptr<Widget> makeWidget() {
    return std::make_unique<Widget>(/* args */);
}
```

Q9. What is std::shared_ptr and what are its costs? [shared_ptr]

Answer

std::shared_ptr implements shared ownership via reference counting. A control block (allocated separately) holds a strong count and a weak count. When the strong count reaches zero the object is destroyed; when the weak count also reaches zero the control block is freed. Costs: atomic reference count increments/decrements (thread-safe but expensive); two pointer indirections for access; larger sizeof than unique_ptr. Use shared_ptr only when shared ownership is genuinely needed.

Example

```
auto sp1 = std::make_shared<std::string>("hello");
std::cout << sp1.use_count(); // 1

{
    auto sp2 = sp1; // shared ownership, count -> 2
    std::cout << sp1.use_count(); // 2
    auto sp3 = sp1; // count -> 3
} // sp2, sp3 destroyed -> count back to 1

std::cout << sp1.use_count(); // 1
// sp1 goes out of scope -> count = 0 -> string destroyed

// make_shared: single allocation for object + control block
// new shared_ptr<T>(new T): TWO allocations
// ALWAYS prefer make_shared
auto fast = std::make_shared<Widget>(); // 1 alloc
auto slow = std::shared_ptr<Widget>(new Widget()); // 2 allocs
```

Q10. What is std::weak_ptr and why is it needed? [weak_ptr]

Answer

std::weak_ptr is a non-owning observer of a shared_ptr-managed object. It does not increment the strong reference count. To access the object, you must call lock() which returns a shared_ptr (empty if the object was already destroyed). It is used to break reference cycles (which would cause memory leaks) and for caches/observers that should not keep objects alive.

Example

```
// Cyclic reference problem:
struct Node {
    std::shared_ptr<Node> next; // CYCLE if A->B->A
};

// Fix with weak_ptr:
struct NodeFixed {
    std::weak_ptr<NodeFixed> next; // no cycle
};

// Observer pattern:
class Cache {
    std::weak_ptr<Resource> cached_;
public:
    std::shared_ptr<Resource> get() {
        if (auto sp = cached_.lock()) {
            return sp;  // object still alive
        }
        // object was destroyed, recreate
        auto fresh = std::make_shared<Resource>();
        cached_ = fresh;
        return fresh;
    }
};
```

Q11. Why should you never call shared_ptr(this) inside a class member function? [shared_ptr] [Pitfalls]

Answer

Creating a shared_ptr from a raw this pointer creates a NEW independent control block. If another shared_ptr already manages this object, you get two control blocks with independent reference counts — when both reach zero, the destructor runs TWICE (double-delete, undefined behavior). The solution is to inherit from std::enable_shared_from_this<T> and call shared_from_this() instead.

Example

```
// WRONG: double-delete!
class Widget {
public:
    std::shared_ptr<Widget> getSelf() {
        return std::shared_ptr<Widget>(this); // NEW control block!
    }
};

// CORRECT: enable_shared_from_this
class Widget : public std::enable_shared_from_this<Widget> {
public:
    std::shared_ptr<Widget> getSelf() {
        return shared_from_this(); // same control block
    }
    // Must be managed by shared_ptr before calling getSelf()!
};

auto w = std::make_shared<Widget>();
auto w2 = w->getSelf(); // safe, same object
```

```
std::cout << w.use_count(); // 2
```

Q12. What are custom deleters and when are they used? [unique_ptr] [Custom Deleter]

Answer

A custom deleter is a callable that replaces delete when the managed object is destroyed. Used for: C API resources that use a function other than free/delete (fclose, SDL_DestroyWindow), objects that must be returned to a pool, resources where destruction requires additional context, or arrays needing special cleanup.

Example

```
// unique_ptr with function-pointer deleter
std::unique_ptr<FILE, int(*)(FILE*)> f(fopen("x.txt","r"), fclose);

// unique_ptr with lambda deleter (smaller code, inlineable)
auto closeFile = [](FILE* f){ if(f) fclose(f); };
std::unique_ptr<FILE, decltype(closeFile)>
    file(fopen("x.txt","r"), closeFile);

// SDL resource management
struct SDLWindowDeleter {
    void operator()(SDL_Window* w) const {
        SDL_DestroyWindow(w);
    }
};
using WindowPtr = std::unique_ptr<SDL_Window, SDLWindowDeleter>;

// Pool deleter
struct PoolReturn {
    Pool* pool;
    void operator()(Widget* w) const { pool->release(w); }
};
std::unique_ptr<Widget, PoolReturn> pw(pool.acquire(), {&pool});
```

Q13. What is the difference between make_unique and make_shared vs using new directly? [Best Practices]

Answer

make_unique/make_shared should always be preferred. Exception safety: smart_ptr<T>(new T()) — if T() throws after new, no leak; but in a function call like f(smart_ptr<T>(new T()), g()) the evaluation order issue in pre-C++17 could cause leaks. make_shared allocates the object and control block in one allocation (better cache locality, one less allocation). make_unique forwards args directly to T's constructor.

Example

```
// PREFER: single allocation, exception-safe
auto sp = std::make_shared<Widget>(arg1, arg2);
auto up = std::make_unique<Widget>(arg1, arg2);

// AVOID: two separate allocations
std::shared_ptr<Widget> sp2(new Widget(arg1, arg2));

// Pre-C++17 exception safety issue:
// evaluate: f(shared_ptr<T>(new T), g())
// Possible order: new T -> g() [throws!] -> shared_ptr ctor
// Result: leak! new T returned but shared_ptr not yet constructed

// make_shared eliminates this:
// f(make_shared<T>(), g())  <-- always safe
```

```
// make_shared downside:
// Object memory freed only when weak_ptr count also hits 0
// (control block and object share one allocation)
```

SECTION 3 — Scope Guards & Lock Management

Q14. What is a scope guard and how do you implement one? [Scope Guard]

Answer

A scope guard is an RAII object that executes an arbitrary callable on destruction, used for cleanup actions that do not fit into a named RAII class. It is the generalization of RAII to arbitrary cleanup. C++23 adds std::scope_exit, scope_success, and scope_fail. Before that, you implement your own or use a library version.

Example

```
template<typename F>
class ScopeGuard {
    F func_;
    bool active_ = true;
public:
    explicit ScopeGuard(F&& f) : func_(std::forward<F>(f)) {}
    ~ScopeGuard() { if (active_) func_(); }
    void dismiss() { active_ = false; }
    ScopeGuard(ScopeGuard&&) = default;
    ScopeGuard(const ScopeGuard&) = delete;
};

// Helper factory
template<typename F>
auto makeScopeGuard(F&& f) {
    return ScopeGuard<std::decay_t<F>>(std::forward<F>(f));
}

void example() {
    auto guard = makeScopeGuard([]{ std::cout << "cleanup!"; });
    if (earlyReturn()) return; // cleanup still runs
    guard.dismiss(); // cancel cleanup on success
}
```

Q15. What is std::lock_guard vs std::unique_lock vs std::scoped_lock? [Mutex] [Lock Management]

Answer

lock_guard: simplest, locks on construction, unlocks on destruction, cannot be manually unlocked, non-movable. unique_lock: more flexible — supports try_lock, timed_lock, manual lock/unlock, deferred locking, and is movable. scoped_lock (C++17): locks multiple mutexes simultaneously using deadlock-avoidance algorithm. Always prefer lock_guard/scoped_lock unless you need unique_lock features.

Example

```
std::mutex m1, m2;

// lock_guard: simplest, prefer this
{
    std::lock_guard<std::mutex> lk(m1);
```

```
    // critical section
} // unlocked here

// unique_lock: manual control
{
    std::unique_lock<std::mutex> lk(m1, std::defer_lock);
    lk.lock();          // manual lock
    lk.unlock();        // manual unlock
    lk.try_lock();      // non-blocking attempt
    // move to another scope:
    auto lk2 = std::move(lk);
}

// scoped_lock: lock TWO mutexes, deadlock-free (C++17)
{
    std::scoped_lock lk(m1, m2); // locks both atomically
} // unlocks both
```

Q16. What is std::shared_lock and when is it used? [Mutex] [Reader-Writer Lock]

Answer

std::shared_lock (C++14, used with std::shared_mutex) implements a reader-writer lock. Multiple readers can hold shared_lock simultaneously. A writer acquires unique_lock (exclusive). This allows concurrent reads while serializing writes — critical for read-heavy workloads like caches, configuration stores, or lookup tables.

Example

```
#include <shared_mutex>

class ThreadSafeCache {
    mutable std::shared_mutex mtx_;
    std::unordered_map<int,std::string> data_;
public:
    std::string get(int key) const {
        std::shared_lock lock(mtx_); // multiple readers OK
        auto it = data_.find(key);
        return it != data_.end() ? it->second : "";
    }

    void put(int key, std::string val) {
        std::unique_lock lock(mtx_); // exclusive write
        data_[key] = std::move(val);
    }
};
```

Q17. What are std::scope_exit, scope_success, and scope_fail (C++23)? [C++23] [Scope Guard]

Answer

C++23 standardises three scope guard types. scope_exit always runs its action on scope exit. scope_success runs only if no exception is propagating (std::uncaught_exceptions() did not increase). scope_fail runs only if an exception is propagating. They replace hand-rolled scope guards and are available now via the <experimental/scope> header in GCC/Clang.

Example

```
#include <scope>  // C++23 or <experimental/scope>

void example() {
    auto always = std::scope_exit([]{ std::cout << "always\n"; });
```

```
    auto onSuccess = std::scope_success([]{ commit(); });

    auto onFail = std::scope_fail([]{ rollback(); });

    doRiskyWork(); // may throw
    // Normal exit:   always + onSuccess run
    // Exception exit: always + onFail run
}

// Database transaction pattern:
void transferFunds(DB& db, int from, int to, int amount) {
    db.begin();
    auto rollbackOnFail = std::scope_fail([&]{ db.rollback(); });
    db.debit(from, amount);  // may throw
    db.credit(to, amount);   // may throw
    db.commit();
}
```

Q18. How do you implement a reentrant lock guard safely? [Mutex] [Advanced]

Answer

std::recursive_mutex allows the same thread to lock it multiple times. Each lock must be matched by an unlock. Use it only when a class method calls another method that also locks the same mutex. Prefer refactoring to avoid recursive locking, as it can hide design issues. std::recursive_timed_mutex adds timed locking.

Example

```
class RecursiveComponent {
    mutable std::recursive_mutex mtx_;
    int count_ = 0;
public:
    void outer() {
        std::lock_guard<std::recursive_mutex> lk(mtx_);
        count_++;
        inner(); // safe: recursive_mutex allows re-entry
    }

    void inner() {
        std::lock_guard<std::recursive_mutex> lk(mtx_); // OK
        count_++;
    }
};

// Better design: separate locked and unlocked implementations
class BetterComponent {
    std::mutex mtx_;
    void innerImpl() { /* logic, no lock */ }
public:
    void outer() {
        std::lock_guard lk(mtx_);
        innerImpl(); // no recursive lock needed
    }
    void inner() {
        std::lock_guard lk(mtx_);
        innerImpl();
    }
};
```

SECTION 4 — Move Semantics & Perfect Forwarding

Q19. What is move semantics and how does it relate to RAII? [Move Semantics]

Answer

Move semantics allow transferring resource ownership from one object to another in O(1) — instead of deep-copying the resource, you steal the pointer/handle and null out the source. This is critical for RAII classes: returning a unique_ptr from a factory, inserting a unique_ptr into a container, or moving a file handle between scopes. Move semantics make RAII classes efficient.

Example

```
class FileHandle {
    int fd_ = -1;
public:
    explicit FileHandle(const char* path)
        : fd_(open(path, O_RDONLY)) {}

    ~FileHandle() { if (fd_ >= 0) close(fd_); }

    // Move: transfer ownership, O(1)
    FileHandle(FileHandle&& o) noexcept : fd_(o.fd_)
    { o.fd_ = -1; } // source left in valid "empty" state

    FileHandle& operator=(FileHandle&& o) noexcept {
        if (this != &o) {
            if (fd_ >= 0) close(fd_);
            fd_ = o.fd_; o.fd_ = -1;
        }
        return *this;
    }

    FileHandle(const FileHandle&) = delete;
    FileHandle& operator=(const FileHandle&) = delete;
};

// Factory: move out of function, O(1)
FileHandle openLog() {
    return FileHandle("/var/log/app.log");
}
```

Q20. What is std::move and what does it actually do? [Move Semantics]

Answer

std::move is a cast — it converts an lvalue to an rvalue reference (xvalue), enabling the move constructor/assignment to be selected by overload resolution. It does NOT actually move anything. The actual transfer happens in the move constructor/operator= that is then invoked. After std::move, the source is in a "valid but unspecified" state — you should not use it except to assign or destroy it.

Example

```
std::string s1 = "hello";
std::string s2 = std::move(s1); // cast to rvalue
// s1 is now valid but empty (typically), s2 has "hello"
std::cout << s2; // "hello"
std::cout << s1.size(); // 0 (implementation-defined but valid)

// DO NOT USE after move:
// std::cout << s1; // valid but unspecified — avoid

// move in containers
```

```
std::vector<std::string> v;
std::string big(1000000, 'x');
v.push_back(std::move(big)); // O(1) — no copy of 1M chars

// In return: DO NOT write return std::move(x);
// The compiler applies NRVO/RVO automatically
std::string makeString() {
    std::string s = "result";
    return s; // NOT std::move(s) — defeats NRVO!
}
```

Q21. What is perfect forwarding and why does it matter for RAII factories? [Perfect Forwarding]

Answer

Perfect forwarding preserves the value category (lvalue/rvalue) and cv-qualifiers of arguments when passing them through a template function. Use std::forward<Args>(args)... in variadic templates. This is what make_unique and make_shared use — they forward constructor arguments directly to T's constructor, enabling in-place construction without any copies.

Example

```
// Without perfect forwarding: always copies
template<typename T, typename Arg>
std::unique_ptr<T> bad_make(Arg arg) {  // arg is lvalue here
    return std::unique_ptr<T>(new T(arg)); // copy of arg
}

// With perfect forwarding: preserves move/copy
template<typename T, typename... Args>
std::unique_ptr<T> my_make_unique(Args&&... args) {
    return std::unique_ptr<T>(
        new T(std::forward<Args>(args)...));
}

class Widget {
    std::string name_;
    std::unique_ptr<Data> data_;
public:
    Widget(std::string n, std::unique_ptr<Data> d)
        : name_(std::move(n)), data_(std::move(d)) {}
};

auto data = std::make_unique<Data>();
// Perfect forwarding: string is moved, unique_ptr is moved
auto w = my_make_unique<Widget>("hello", std::move(data));
```

Q22. Why should move constructors and move assignment operators be marked noexcept? [Move Semantics] [noexcept]

Answer

std::vector::push_back will use the move constructor during reallocation ONLY if it is marked noexcept. If the move constructor can throw, vector uses the copy constructor as a fallback (to maintain strong exception safety). A throwing move is almost always a design error — moves should only transfer pointers/handles and set the source to null, which cannot fail. Mark them noexcept.

Example

```
class HeavyObject {
    std::vector<double> data_;
public:
    HeavyObject(size_t n) : data_(n) {}
```

```
    // noexcept: vector uses this during reallocation
    HeavyObject(HeavyObject&& o) noexcept
        : data_(std::move(o.data_)) {}

    HeavyObject& operator=(HeavyObject&& o) noexcept {
        data_ = std::move(o.data_);
        return *this;
    }
};

// Verify:
static_assert(std::is_nothrow_move_constructible_v<HeavyObject>);

// Without noexcept: push_back falls back to copy (O(n)!)
std::vector<HeavyObject> v;
v.reserve(10);
v.push_back(HeavyObject(1000)); // move used (noexcept declared)
```

Q23. What happens if you throw from a destructor during stack unwinding? [Exception Safety] [Destructor]

Answer

If a destructor throws while another exception is already propagating (std::uncaught_exceptions() > 0), std::terminate() is called immediately — the program crashes. Destructors must NEVER propagate exceptions. If a destructor operation can fail, either swallow the exception (log it), or provide a separate close()/flush() function that can throw, and have the destructor call it in a try/catch.

Example

```
class DangerousFile {
    bool dirty_ = true;
public:
    // BAD: destructor may throw
    ~DangerousFile() {
        flush(); // MAY THROW -- UNDEFINED BEHAVIOR during unwind!
    }
};

// GOOD: separate function for fallible cleanup
class SafeFile {
    FILE* f_;
    bool dirty_ = true;
public:
    // Call this explicitly when you can handle errors
    void flush() {
        if (dirty_) {
            if (fflush(f_) != 0)
                throw std::runtime_error("flush failed");
            dirty_ = false;
        }
    }
    // Destructor: best-effort, swallow exceptions
    ~SafeFile() {
        try { flush(); }
        catch (...) { /* log, but do not propagate */ }
        fclose(f_);
    }
};
```

SECTION 5 — Memory Ownership Patterns

Q24. When should you use raw pointers in modern C++? [Ownership] [Best Practices]

Answer

Raw pointers should be used only as NON-OWNING observers — when you need to point to something but do not own it or manage its lifetime. Passing a raw pointer or reference to a function that uses but does not store it is fine. Never use raw pointers to express ownership — use unique_ptr (sole ownership) or shared_ptr (shared ownership) instead.

Example

```
// RAW POINTER OK: non-owning parameter
void processWidget(const Widget* w) { // just reads, no ownership
    if (w) w->draw();
}

// RAW POINTER OK: non-owning member (lifetime guaranteed by caller)
class Renderer {
    const Scene* scene_; // does NOT own — Scene outlives Renderer
public:
    explicit Renderer(const Scene* s) : scene_(s) {}
    void render() { scene_->draw(); }
};

// OWNERSHIP: always use smart pointers
class App {
    std::unique_ptr<Widget> widget_; // owns
    std::shared_ptr<Config> config_; // shared ownership
public:
    void setWidget(std::unique_ptr<Widget> w) {
        widget_ = std::move(w);
    }
    Widget* getWidget() { return widget_.get(); } // borrow only
};
```

Q25. What is the pimpl idiom and how does RAII enable it? [Pimpl] [ABI Stability]

Answer

Pimpl (Pointer to Implementation) hides implementation details behind a forward-declared private pointer, reducing compile-time dependencies and providing ABI stability. RAII via unique_ptr makes pimpl safe and leak-proof without manual delete. The forward declaration requires the destructor to be defined in the .cpp file where Impl is complete.

Example

```
// widget.h
class Widget {
    struct Impl;
    std::unique_ptr<Impl> pImpl_;
public:
    explicit Widget(int x);
    ~Widget();  // must declare, define in .cpp
    Widget(Widget&&) noexcept;
    Widget& operator=(Widget&&) noexcept;
    void draw() const;
};

// widget.cpp
struct Widget::Impl {
    int x;
```

```
    void draw() const { /* heavy implementation */ }
};

Widget::Widget(int x) : pImpl_(std::make_unique<Impl>(x)) {}
Widget::~Widget() = default;  // defined here: Impl is complete
Widget::Widget(Widget&&) noexcept = default;
Widget& Widget::operator=(Widget&&) noexcept = default;
void Widget::draw() const { pImpl_->draw(); }
```

Q26. What is std::pmr and how do polymorphic memory resources relate to RAII? [Memory Resources] [C++17]

Answer

std::pmr (C++17, <memory_resource>) provides a runtime-polymorphic allocator interface. Memory resources like monotonic_buffer_resource (arena allocator), unsynchronized_pool_resource, and synchronized_pool_resource implement std::pmr::memory_resource. They themselves are RAII objects. std::pmr containers use them, enabling stack allocation, pool allocation, or custom strategies without changing container types.

Example

```
#include <memory_resource>

void processRequest(const Request& req) {
    // Stack-backed arena: all allocations come from stack buffer
    std::byte buf[4096];
    std::pmr::monotonic_buffer_resource arena{buf, sizeof(buf)};

    // RAII: arena freed when function exits
    std::pmr::vector<std::pmr::string> items{&arena};
    for (auto& s : req.data)
        items.emplace_back(s, &arena); // no heap!

    std::pmr::unordered_map<
        std::pmr::string, int> freq{&arena};
    for (auto& item : items) freq[item]++;
    // All freed at end of scope: arena dtored -> stack unwound
}
```

Q27. What is a handle-body (or envelope-letter) pattern and how does it use RAII? [Design Pattern]

Answer

The handle-body pattern separates interface (handle) from implementation (body). The handle is a lightweight value-semantic object that holds a shared_ptr or unique_ptr to the body. This gives value semantics on the outside (copyable, assignable) while hiding the potentially expensive or polymorphic implementation inside. The smart pointer manages the body lifetime via RAII.

Example

```
class Image {
    struct Body {
        int width, height;
        std::vector<uint8_t> pixels;
        Body(int w, int h)
            : width(w), height(h), pixels(w*h*4) {}
    };
    std::shared_ptr<Body> body_; // lazy sharing
public:
    Image(int w, int h)
        : body_(std::make_shared<Body>(w,h)) {}
```

```
    // Copy: shallow (shared body)
    Image(const Image&) = default;

    // Copy-on-write before mutation
    void setPixel(int x, int y, uint8_t r, uint8_t g, uint8_t b) {
        if (!body_.unique()) // not sole owner
            body_ = std::make_shared<Body>(*body_); // deep copy
        auto& p = body_->pixels;
        p[(y * body_->width + x) * 4 + 0] = r;
    }
};
```

Q28. How do you safely store RAII objects in STL containers? [STL] [Ownership]

Answer

Unique ownership (unique_ptr) can be stored in containers because unique_ptr is movable. Use emplace_back or push_back(std::move(...)). You cannot copy unique_ptrs into containers. shared_ptr is fully copyable and movable. Non-movable RAII types (custom mutexes, streams) must be managed via pointers. For in-place construction, use emplace with factory arguments.

Example

```
// unique_ptr in vector
std::vector<std::unique_ptr<Widget>> widgets;
widgets.push_back(std::make_unique<Widget>(1));
widgets.emplace_back(std::make_unique<Widget>(2));

// Cannot copy unique_ptr:
// auto w = widgets[0]; // ERROR
auto& ref = widgets[0]; // OK: reference
Widget* raw = widgets[0].get(); // OK: borrow

// shared_ptr in map
std::map<int, std::shared_ptr<Resource>> cache;
cache[1] = std::make_shared<Resource>("db");
auto r = cache[1]; // shared ownership, count++

// Non-copyable, non-movable: use unique_ptr wrapper
std::vector<std::unique_ptr<std::mutex>> mutexes;
mutexes.push_back(std::make_unique<std::mutex>());
```

SECTION 6 — Advanced RAII Patterns

Q29. What is std::unique_ptr with a custom deleter for handle types? [Custom Deleter] [C API]

Answer

Many C APIs use opaque handle types and dedicated destroy functions. Wrapping them with unique_ptr + custom deleter gives RAII safety with zero overhead. The deleter type becomes part of unique_ptr's type, so different deleters produce different types. Use a function-pointer deleter or a small functor.

Example

```
// OpenSSL RAII wrappers
struct EVP_CTX_Deleter {
```

```
    void operator()(EVP_MD_CTX* ctx) const {
        EVP_MD_CTX_free(ctx);
    }
};
using EVPContext = std::unique_ptr<EVP_MD_CTX, EVP_CTX_Deleter>;

// Vulkan RAII wrapper
struct VkDeviceDeleter {
    VkInstance instance;
    PFN_vkDestroyDevice destroyFn;
    void operator()(VkDevice dev) const {
        destroyFn(dev, nullptr);
    }
};
using VkDevicePtr = std::unique_ptr<
    std::remove_pointer_t<VkDevice>, VkDeviceDeleter>;

// Null deleter (for non-owning unique_ptr)
struct NoDelete { void operator()(void*) const {} };
```

Q30. What is the copy-and-swap idiom and why does it simplify RAII classes? [Copy-and-Swap] [Design Pattern]

Answer

Copy-and-swap implements copy assignment by making a copy of the right-hand side (via the copy constructor), then swapping the internals with the current object, and letting the old internals be destroyed in the local copy's destructor. This achieves: strong exception safety (copy may throw, but swap and destruction cannot), self-assignment safety, and code reuse between copy and move operations.

Example

```
class Buffer {
    char* data_; size_t size_;
public:
    explicit Buffer(size_t n)
        : data_(new char[n]), size_(n) {}

    ~Buffer() { delete[] data_; }

    // Copy constructor
    Buffer(const Buffer& o)
        : data_(new char[o.size_]), size_(o.size_)
    { std::copy(o.data_, o.data_+o.size_, data_); }

    // swap: no-throw
    friend void swap(Buffer& a, Buffer& b) noexcept {
        using std::swap;
        swap(a.data_, b.data_);
        swap(a.size_, b.size_);
    }

    // Unified assignment (handles both copy and move)
    Buffer& operator=(Buffer other) noexcept { // copy in param
        swap(*this, other); // no-throw
        return *this;
    } // old data destroyed in other's dtor
};
```

Q31. How do you implement a RAII-based thread that joins automatically? [Thread] [RAII]

Answer

std::thread does not join on destruction — if a thread is joinable when destroyed, std::terminate() is called. You must either join() or detach() before destruction. An RAII thread wrapper (similar to C++20 std::jthread) joins automatically in the destructor. C++20 adds std::jthread which also supports stop tokens.

Example

```
// Pre-C++20: RAII thread wrapper
class JThread {
    std::thread t_;
public:
    template<typename F, typename... Args>
    explicit JThread(F&& f, Args&&... args)
        : t_(std::forward<F>(f), std::forward<Args>(args)...) {}

    ~JThread() {
        if (t_.joinable()) t_.join(); // always joins
    }

    JThread(JThread&&) = default;
    JThread(const JThread&) = delete;
};

void example() {
    JThread t([]{ doWork(); }); // launches thread
    // ... do other things ...
    // thread joined here automatically
}

// C++20: std::jthread
std::jthread jt([](std::stop_token st) {
    while (!st.stop_requested()) doWork();
});
// jt.request_stop() + jt joins on destruction
```

Q32. What is a poison-pill or flag pattern with RAII for cancellation? [Thread] [Cancellation]

Answer

A cancellation token or stop flag is an RAII object that signals a thread to stop when the owning scope exits. The destructor sets the flag and waits for the thread to notice. C++20 std::stop_token / std::stop_source standardise this pattern. For pre-C++20 code, a shared atomic<bool> with a RAII setter is the idiom.

Example

```
class CancelScope {
    std::atomic<bool> cancelled_{false};
    std::thread worker_;
public:
    template<typename F>
    explicit CancelScope(F&& work) {
        worker_ = std::thread([this, work=std::forward<F>(work)]{
            work(cancelled_);
        });
    }
    ~CancelScope() {
        cancelled_.store(true);
        if (worker_.joinable()) worker_.join();
    }
};

{
    CancelScope scope([](std::atomic<bool>& done){
        while (!done.load()) {
            processBatch();
```

```
        }
    });
    // do work in main thread...
} // worker cancelled and joined here
```

Q33. How does RAII apply to transaction scopes in databases? [Transaction] [Design Pattern]

Answer

A transaction RAII object begins a transaction on construction and rolls back on destruction unless commit() was called. This ensures no transaction is ever left open (which would block other writers) and that partial work is never committed. The destructor must not throw — catch and log any rollback failure.

Example

```
class DbTransaction {
    DbConnection& conn_;
    bool committed_ = false;
public:
    explicit DbTransaction(DbConnection& c) : conn_(c) {
        conn_.execute("BEGIN");
    }

    void commit() {
        conn_.execute("COMMIT");
        committed_ = true;
    }

    ~DbTransaction() {
        if (!committed_) {
            try { conn_.execute("ROLLBACK"); }
            catch (...) { /* log: cannot propagate */ }
        }
    }
};

void transferFunds(DbConnection& db, int from, int to, int amt) {
    DbTransaction tx(db);
    db.execute("UPDATE accounts SET bal=bal-? WHERE id=?",amt,from);
    db.execute("UPDATE accounts SET bal=bal+? WHERE id=?",amt,to);
    tx.commit(); // only committed if both succeed
} // rolled back automatically on any exception
```

Q34. What is the finally-like pattern and how do defer-style designs compare to RAII? [Design Pattern] [Comparison]

Answer

Go's defer and Java/Python's try/finally execute cleanup at function exit. RAII is more powerful: cleanup is lexically scoped to object lifetime, not function exit, so RAII objects inside loops or conditionals clean up at the right moment. Scope guards simulate defer-style. RAII is also composable — RAII members in RAII classes create automatic cleanup chains without any explicit finally blocks.

Example

```
// "defer" simulation with scope guard
void processFile(const char* path) {
    FILE* f = fopen(path, "r");
    if (!f) throw std::runtime_error("open failed");

    // "defer fclose(f)" — runs on any exit from this scope
    auto defer_close = makeScopeGuard([&]{ fclose(f); });
```

```
    // RAII more powerful: cleanup at block exit, not just function
    for (int i = 0; i < 10; ++i) {
        std::unique_ptr<char[]> buf(new char[4096]);
        fread(buf.get(), 1, 4096, f);
        process(buf.get());
        // buf freed at END OF LOOP BODY each iteration
    }
    // defer_close runs here
}
```

Q35. What is std::unique_ptr::release() and when is it safe to use? [unique_ptr] [Ownership Transfer]

Answer

release() relinquishes ownership — returns the raw pointer and sets the unique_ptr to null WITHOUT calling the deleter. Use it when you need to transfer ownership to a C API that takes a raw pointer and promises to free it, or when constructing a shared_ptr from a unique_ptr. Calling release() without capturing the returned pointer is a guaranteed leak.

Example

```
auto up = std::make_unique<Widget>();

// Transfer to C API that owns the pointer
Widget* raw = up.release(); // up is now null, no delete called
c_api_take_ownership(raw);  // C API will free it

// Transfer to shared_ptr
auto up2 = std::make_unique<Widget>();
std::shared_ptr<Widget> sp(up2.release()); // up2 null, sp owns

// WRONG: leak!
// up.release(); // returned pointer discarded -> leak!

// CORRECT pattern when converting:
auto up3 = std::make_unique<Widget>();
std::shared_ptr<Widget> sp2 = std::move(up3);
// Preferred: move directly, no release needed
```

Q36. How do you implement a RAII wrapper for OpenGL / Vulkan resources? [GPU Resources] [C API]

Answer

Graphics APIs use integer handles (GLuint, VkImage) with paired create/destroy functions. Wrapping them in RAII classes prevents resource leaks common in graphics code. The deleter must not throw. For Vulkan, the device handle must be available at destruction time — store it in the deleter.

Example

```
// OpenGL texture RAII
class GLTexture {
    GLuint id_ = 0;
public:
    GLTexture() { glGenTextures(1, &id_); }
    ~GLTexture() { glDeleteTextures(1, &id_); }
    GLTexture(GLTexture&& o) noexcept : id_(o.id_) { o.id_=0; }
    GLTexture& operator=(GLTexture&& o) noexcept {
        glDeleteTextures(1, &id_);
        id_ = o.id_; o.id_ = 0; return *this;
    }
    GLTexture(const GLTexture&) = delete;
    GLTexture& operator=(const GLTexture&) = delete;
```

```
    GLuint id() const { return id_; }
};

// Vulkan buffer with device-aware deleter
struct VkBufferDeleter {
    VkDevice device;
    void operator()(VkBuffer* buf) const {
        vkDestroyBuffer(device, *buf, nullptr);
        delete buf;
    }
};
using VkBufferPtr = std::unique_ptr<VkBuffer, VkBufferDeleter>;
```

Q37. What is the "poison pill" destructor pattern for catching use-after-move? [Debugging] [Move Semantics]

Answer

After being moved-from, an RAII object is in a valid but unspecified state. Debug builds can set a "moved-from" flag in the move constructor and assert in all other operations that the object has not been used after being moved. This catches use-after-move bugs early in development without overhead in release builds.

Example

```
class Resource {
    int* data_;
#ifndef NDEBUG
    bool moved_from_ = false;
#endif
    void checkValid() const {
#ifndef NDEBUG
        assert(!moved_from_ && "use-after-move detected!");
#endif
    }
public:
    explicit Resource(int n) : data_(new int(n)) {}
    ~Resource() { delete data_; }

    Resource(Resource&& o) noexcept : data_(o.data_) {
        o.data_ = nullptr;
#ifndef NDEBUG
        o.moved_from_ = true;
#endif
    }

    int value() const { checkValid(); return *data_; }
};
```

SECTION 7 — RAII Pitfalls, Anti-Patterns & Diagnostics

Q38. What is the "zombie object" problem and how does RAII prevent it? [Pitfalls]

Answer

A zombie object is one that has been partially constructed (constructor threw after some resources were acquired) or is in an invalid post-move state. RAII prevents zombies by ensuring that anything acquired before

the throw in the constructor is managed by RAII sub-objects (which are destroyed during stack unwinding). The destructor of a partially-constructed object is NOT called — only fully-constructed member sub-objects are destroyed.

Example

```
// DANGEROUS: partial construction leaves zombie
class Bad {
    FILE* f_ = nullptr;
    int*  buf_ = nullptr;
public:
    Bad() {
        f_ = fopen("x","r");
        buf_ = new int[1000]; // if this throws...
        // f_ leaks! Bad::~Bad() not called!
    }
    ~Bad() { fclose(f_); delete[] buf_; }
};

// SAFE: RAII members clean up during partial construction
class Good {
    // Both are RAII -- if unique_ptr<int[]> throws,
    // FileGuard dtor still runs
    FileGuard file_;
    std::unique_ptr<int[]> buf_;
public:
    Good()
        : file_("x"),           // RAII acquire
          buf_(new int[1000]) { // if throws, file_ dtor runs
    }
};
```

Q39. What is a cyclic shared_ptr and how does it cause memory leaks? [shared_ptr] [Memory Leak] [Pitfalls]

Answer

A cycle occurs when two objects each hold a shared_ptr to each other. The reference counts never reach zero because each object keeps the other alive. The objects are never destroyed even when no external shared_ptr exists. Fix: break cycles by making one direction a weak_ptr. The choice of which direction to weaken is a design decision based on ownership semantics.

Example

```
// LEAK: cyclic reference
struct A {
    std::shared_ptr<B> b; // A -> B
};
struct B {
    std::shared_ptr<A> a; // B -> A (CYCLE!)
};

{
    auto a = std::make_shared<A>();
    auto b = std::make_shared<B>();
    a->b = b; // a.use_count=1, b.use_count=2
    b->a = a; // a.use_count=2, b.use_count=2
} // a,b go out of scope: counts drop to 1 -- NEVER ZERO!
// LEAK: both objects kept alive by each other

// FIX: weak_ptr in child->parent direction
struct BFixed {
    std::weak_ptr<A> a; // does not increment count
};
```

Q40. What are common anti-patterns that defeat RAII? [Anti-Patterns] [Pitfalls]

Answer

Common anti-patterns: (1) Storing a raw pointer to a resource and deleting it manually in the destructor — forgetting to handle copies properly. (2) Two-phase initialization (default construct + init()) — the object exists in an invalid state. (3) Disabling copy and move without documenting it. (4) Catching all exceptions in a constructor and not cleaning up acquired resources. (5) Calling virtual functions from constructors when RAII sub-objects have not yet been constructed.

Example

```
// ANTI-PATTERN 1: manual delete in dtor
class Bad1 {
    int* data_;
public:
    Bad1() : data_(new int[100]) {}
    ~Bad1() { delete[] data_; }
    // Missing copy ctor/assignment -> double-delete!
};

// ANTI-PATTERN 2: two-phase init
class Bad2 {
    bool initialized_ = false;
public:
    Bad2() {} // invalid state!
    void init() { initialized_ = true; }
    void use() { assert(initialized_); }
    // RAII guarantee broken: resource not acquired in ctor
};

// CORRECT: single-phase, invariant in constructor
class Good2 {
    std::vector<int> data_;
public:
    explicit Good2(size_t n) : data_(n) {} // always valid
    void use() { /* data_ always valid */ }
};
```

Q41. How does address sanitizer (ASan) help detect RAII violations? [Diagnostics] [Tooling]

Answer

AddressSanitizer instruments heap allocations/deallocations and detects: use-after-free (accessing a deleted RAII-managed resource), double-free (destructor called twice due to shallow copy), heap-buffer-overflow, and use-after-return. Compile with -fsanitize=address -g. LeakSanitizer (LSan) detects memory leaks from RAII objects not properly destroying their resources.

Example

```
// Compile: g++ -fsanitize=address -g -o prog prog.cpp

// Use-after-free: caught by ASan
Widget* raw = nullptr;
{
    auto up = std::make_unique<Widget>();
    raw = up.get();
} // up destroyed -> Widget freed
raw->draw(); // ASan: heap-use-after-free!

// Double-free: caught by ASan
class BadCopy {
    int* p_;
public:
```

```
    BadCopy() : p_(new int(0)) {}
    ~BadCopy() { delete p_; }
    // No copy constructor -> shallow copy -> double free
};
BadCopy a;
BadCopy b = a; // shallow copy!
// both dtors: delete same pointer -> ASan: double-free!
```

Q42. What is the "self-assignment" problem in RAII classes and how is it handled? [Assignment] [Pitfalls]

Answer

Self-assignment (a = a) is legal. A naive copy-assignment that frees the resource first then copies from the source will free the data it is about to read — a subtle bug. The copy-and-swap idiom handles self-assignment correctly because the copy is made before any modification. Alternatively, check for self-assignment with an identity test before proceeding.

Example

```
// WRONG: self-assignment corrupts data
Buffer& operator=(const Buffer& o) {
    delete[] data_;           // frees data_
    size_ = o.size_;
    data_ = new char[size_];
    std::copy(o.data_,...);   // reads freed memory if o==*this!
    return *this;
}

// FIX 1: identity guard
Buffer& operator=(const Buffer& o) {
    if (this != &o) { // guard
        char* newData = new char[o.size_];
        std::copy(o.data_, o.data_+o.size_, newData);
        delete[] data_;
        data_ = newData;
        size_ = o.size_;
    }
    return *this;
}

// FIX 2: copy-and-swap (handles self-assignment automatically)
Buffer& operator=(Buffer o) { // o is a copy
    swap(*this, o); return *this; // old data freed in o's dtor
}
```

Q43. How do you detect and fix a resource leak in a constructor that throws? [Exception Safety] [Constructor]

Answer

If a constructor throws, the destructor is NOT called. Any resources acquired before the throw must be managed by RAII sub-objects (member variables with their own destructors) rather than raw pointers. If you must use raw resources in a constructor, acquire them in a specific order and wrap each in an RAII type before the next acquisition.

Example

```
// LEAK: f_ not freed if second allocation throws
class Broken {
    FILE* f_;
    char* buf_;
public:
```

```
    Broken() : f_(fopen("x","r")), buf_(new char[BIG]) {}
    ~Broken() { fclose(f_); delete[] buf_; }
    // If new char[BIG] throws: ~Broken() not called, f_ leaks!
};

// FIXED: RAII members guarantee cleanup
class Fixed {
    struct FileClose { void operator()(FILE*f){fclose(f);} };
    std::unique_ptr<FILE, FileClose> f_;
    std::unique_ptr<char[]> buf_;
public:
    Fixed()
        : f_(fopen("x","r")),   // RAII
          buf_(new char[BIG]) { // if throws: f_ dtor runs!
    }
};
```

SECTION 8 — Modern C++ RAII & Expert Topics

Q44. How does std::optional interact with RAII types? [optional] [C++17]

Answer

std::optional<T> conditionally holds a T. When the optional is disengaged (empty), T's destructor is NOT called. When the optional goes out of scope while holding a value, T's destructor IS called. This makes optional safe for RAII types — the resource is acquired when optional is assigned and released when the optional is destroyed or reset().

Example

```
std::optional<FileHandle> maybeFile;

if (shouldOpen) {
    maybeFile.emplace("data.txt"); // constructs in-place
    // FileHandle acquired only if condition is true
}

// Reset releases the resource
maybeFile.reset(); // ~FileHandle() called, file closed

// Assign new value: old resource released, new acquired
maybeFile = FileHandle("other.txt");

// optional goes out of scope: if it holds a value, dtor called

// Use with RAII lock:
std::optional<std::lock_guard<std::mutex>> maybeLock;
if (needsLock) maybeLock.emplace(myMutex);
// NOTE: lock_guard is not movable, optional<lock_guard> works
// because emplace constructs in-place
```

Q45. How does RAII compose with coroutines in C++20? [Coroutines] [C++20]

Answer

RAII objects work correctly across co_await suspension points — their destructors run when the coroutine frame is destroyed (either on completion or on the coroutine handle's destruction). However, holding mutexes across co_await is dangerous because the coroutine may resume on a different thread. Use co_await only when the lock is NOT held, or use a specialized async_lock awaitable.

Example

```
// SAFE: unique_ptr lives across co_await
Task<void> processAsync() {
    auto data = std::make_unique<Data>(); // RAII acquired
    co_await asyncReadInto(data.get());   // suspension OK
    co_await asyncWrite(data.get());      // suspension OK
    // data freed when coroutine completes or is destroyed
}

// DANGEROUS: holding mutex across co_await
Task<void> badLocking(std::mutex& m) {
    std::lock_guard lk(m);    // locks
    co_await someAwaitable(); // resumes on different thread!
    // mutex still held -- possibly from wrong thread!
}

// SAFE: release lock before await
Task<void> goodLocking(std::mutex& m, Data& d) {
    { std::lock_guard lk(m); d.prepare(); } // lock released
    co_await someAwaitable(); // no lock held
    { std::lock_guard lk(m); d.finalize(); }
}
```

Q46. What is std::jthread (C++20) and how does it improve on std::thread RAII? [Thread] [C++20]

Answer

std::jthread automatically joins on destruction (unlike std::thread which calls terminate() if joinable when destroyed). It also integrates with std::stop_token/std::stop_source for cooperative cancellation. The thread function receives a stop_token as the first argument and can periodically check stop_requested(). The owning jthread requests a stop and waits in its destructor.

Example

```
#include <thread>
#include <stop_token>

// std::jthread: auto-join + stop support
{
    std::jthread worker([](std::stop_token st) {
        while (!st.stop_requested()) {
            processNextItem();
            std::this_thread::sleep_for(10ms);
        }
        std::cout << "worker stopping gracefully\n";
    });

    std::this_thread::sleep_for(1s);
    worker.request_stop(); // signal cancellation
    // worker.join() called automatically in dtor
}

// Compare: std::thread would call terminate() here:
// std::thread t([]{while(true)processNextItem();});
// } -- terminate()! thread still joinable
```

Q47. What is the "type-erased deleter" pattern for RAII with runtime polymorphism? [Type Erasure] [Advanced]

Answer

shared_ptr stores the deleter in its control block as a type-erased function — the deleter type does not appear in the shared_ptr type. This allows: storing shared_ptrs to related types in the same container, passing shared_ptrs through type-erased interfaces, and converting from unique_ptr (which encodes the deleter in its type) to shared_ptr.

Example

```
// unique_ptr: deleter in type
using FilePtr = std::unique_ptr<FILE, int(*)(FILE*)>;
FilePtr a(fopen("a","r"), fclose);
FilePtr b(fopen("b","r"), fclose);
// std::vector<FilePtr> ok -- same type

// shared_ptr: deleter type-erased
std::vector<std::shared_ptr<void>> resources;

resources.push_back(
    std::shared_ptr<FILE>(fopen("x","r"), fclose));

resources.push_back(
    std::shared_ptr<Widget>(new Widget(),
        [](Widget* w){ w->cleanup(); delete w; }));

// Different types, different deleters -- same vector!
// When each shared_ptr is destroyed, correct deleter called

// Convert unique_ptr -> shared_ptr (inherits deleter)
auto up = std::unique_ptr<Widget, CustomDel>(new Widget());
std::shared_ptr<Widget> sp = std::move(up); // deleter preserved
```

Q48. How do you implement RAII for a stack of operations (undo/rollback)? [Undo] [Design Pattern]

Answer

An undo stack RAII object pushes an undo action on construction and pops/executes it on destruction unless committed. This is useful for UI operations, editor commands, and multi-step algorithms. The destructor must not throw. Stack of std::function<void()> or a vector of Command objects are common implementations.

Example

```
class UndoScope {
    std::vector<std::function<void()>> undos_;
    bool committed_ = false;
public:
    void addUndo(std::function<void()> f) {
        undos_.push_back(std::move(f));
    }
    void commit() { committed_ = true; }
    ~UndoScope() {
        if (!committed_) {
            // Execute undos in reverse order
            for (auto it = undos_.rbegin();
                 it != undos_.rend(); ++it) {
                try { (*it)(); }
                catch (...) { /* log */ }
            }
        }
    }
};
```

```
void complexOperation(Editor& ed) {
    UndoScope undo;
    ed.insertText("Hello"); undo.addUndo([&]{ed.deleteText();});
    ed.setFont(Bold);       undo.addUndo([&]{ed.setFont(Normal);});
    if (userCancels()) return; // undos execute in reverse
    undo.commit();
}
```

Q49. What is the intrusive reference counting pattern and how does it compare to shared_ptr? [Reference Counting] [Advanced]

Answer

Intrusive reference counting stores the count inside the object itself (not a separate control block). Benefits: smaller memory footprint, better cache locality (object + count together), ability to create shared_ptr from raw pointer at any time (via enable_shared_from_this or Boost intrusive_ptr). Downside: the object must inherit from or embed the reference counter. Boost::intrusive_ptr and Qt's QSharedData use this pattern.

Example

```
// Intrusive ref-count base
class RefCounted {
    mutable std::atomic<int> refCount_{0};
public:
    void addRef() const { refCount_.fetch_add(1); }
    void release() const {
        if (refCount_.fetch_sub(1) == 1)
            delete this; // self-delete when count hits 0
    }
protected:
    virtual ~RefCounted() = default;
};

template<typename T>
class IntrusivePtr {
    T* ptr_ = nullptr;
public:
    explicit IntrusivePtr(T* p) : ptr_(p) { if(ptr_) ptr_->addRef(); }
    ~IntrusivePtr() { if(ptr_) ptr_->release(); }
    IntrusivePtr(const IntrusivePtr& o) : ptr_(o.ptr_)
    { if(ptr_) ptr_->addRef(); }
    IntrusivePtr(IntrusivePtr&& o) noexcept : ptr_(o.ptr_)
    { o.ptr_ = nullptr; }
    T* get() const { return ptr_; }
    T* operator->() const { return ptr_; }
};
```

Q50. How do you write a move-only RAII type that works correctly with std::vector reallocation? [Move Semantics] [noexcept]

Answer

std::vector uses the move constructor during reallocation if and only if it is noexcept. If not noexcept, it falls back to copy (which may fail for move-only types, causing a compile error). So a move-only RAII type must have a noexcept move constructor to work in a vector. Also ensure the moved-from state is safe to destroy.

Example

```
class Connection {
    int socket_ = -1;
public:
    explicit Connection(const char* host)
        : socket_(connectTo(host)) {}
```

```
    ~Connection() {
        if (socket_ >= 0) ::close(socket_);
    }

    // noexcept: vector can move during reallocation
    Connection(Connection&& o) noexcept
        : socket_(std::exchange(o.socket_, -1)) {}

    Connection& operator=(Connection&& o) noexcept {
        if (this != &o) {
            if (socket_ >= 0) ::close(socket_);
            socket_ = std::exchange(o.socket_, -1);
        }
        return *this;
    }

    Connection(const Connection&) = delete;
    Connection& operator=(const Connection&) = delete;
};

static_assert(std::is_nothrow_move_constructible_v<Connection>);
std::vector<Connection> pool; // works: noexcept move used
```